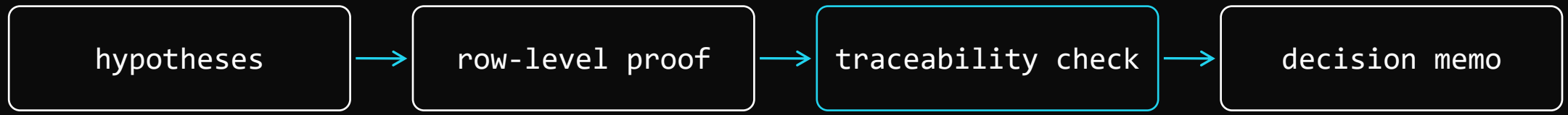
Open Agent Hackathon 2026 · GenAI.Works / HackOS

VeriAgent Why

the agent that explains why a KPI dropped



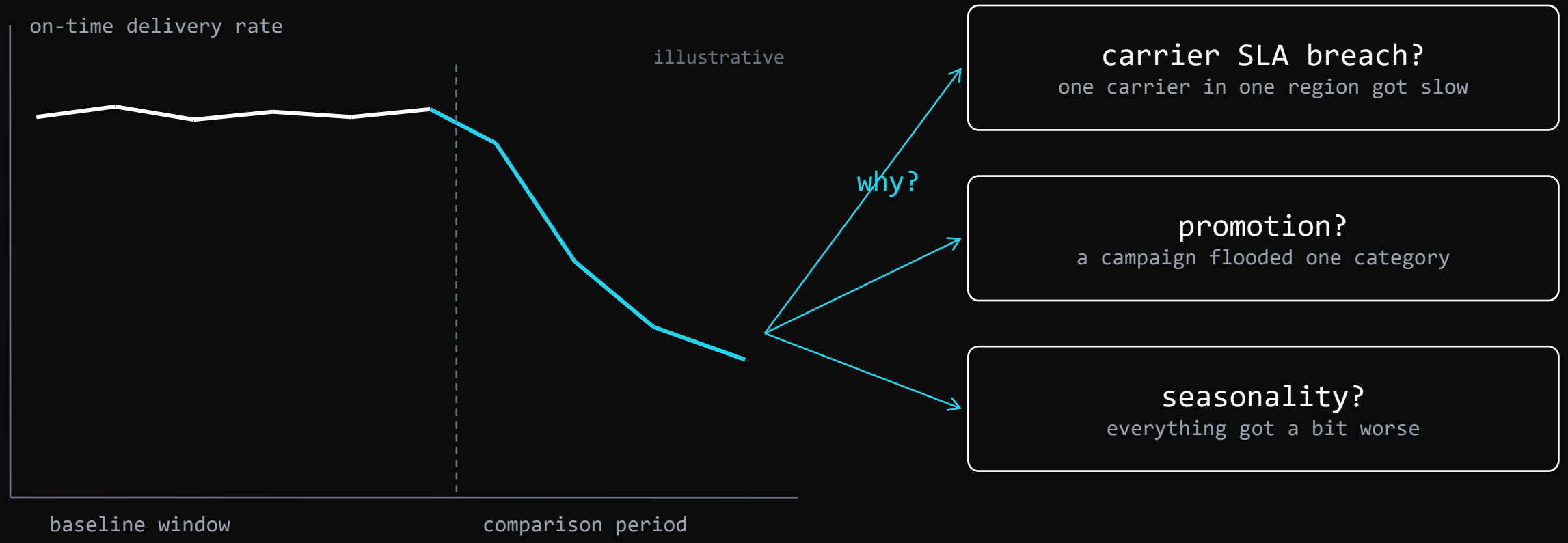
Track 02 · “The Agent That Can Explain Why”

Alonso Alvira Ballano (solo)

github.com/AlonsoAlviraa/veriagent-why

A KPI drops, and the first plausible story wins

Target user: an e-commerce ops analyst who must explain the drop before the meeting



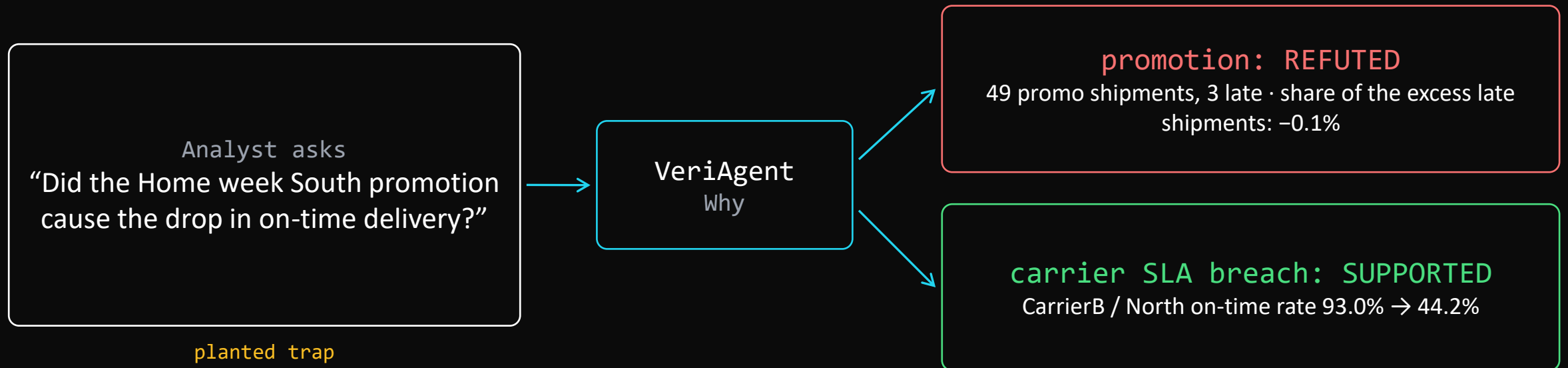
ad-hoc SQL under time pressure

the first plausible story wins

leading questions get confirmed

Ask a leading question, get the data's answer

Tests competing causes against Zetaris data · every claim cites its rows · confidence is a formula

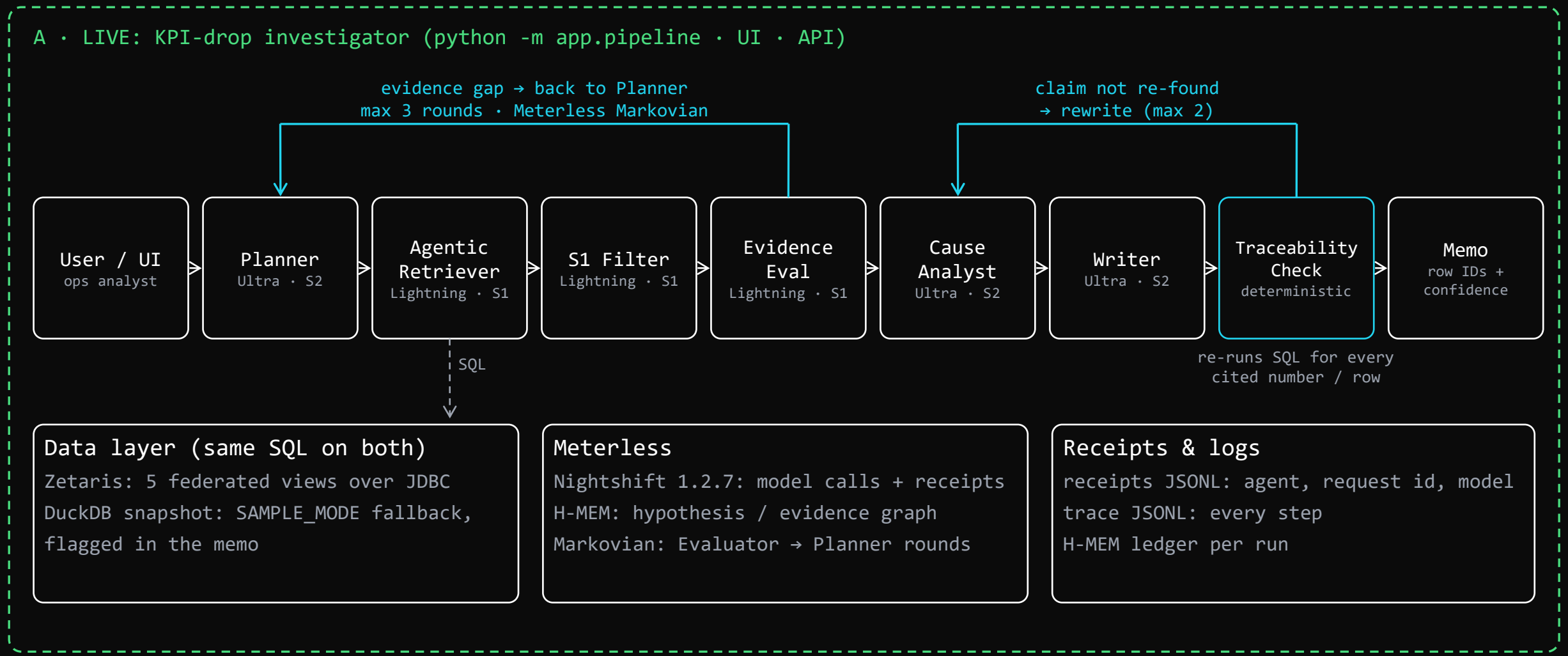


confidence 100	2 / 2 claims pass	row click-through
round(100·(0.45C + 0.35A + 0.20(1−R)))	Traceability Check	v_logistics row S000977

Live run, 26 Oct 2026: Zetaris JDBC + Meterless Nightshift 1.2.7 (output_examples/example_5_live_zetaris_nightshift.json). Synthetic data, planted promotion trap.

Six agents, one deterministic check, two feedback loops

Why did this KPI drop? hypotheses · row-level proof · traceability check · decision memo



Green = runs in the product today · Amber = benchmark harness only

Zetaris is the data layer: five federated views, one SQL path

Retriever, Evidence Evaluator and Traceability Check all query the same views over JDBC

live data path (Zetaris Lightning 2.4.3, local hackathon platform)

PostgreSQL 16
veriagent_pg
v_orders · v_logistics
v_returns

REST JSON source
veriagent_rest
v_tickets · v_promotions

Zetaris SchemaStore
container veriagent

v_orders	1,818 rows
v_logistics	1,818 rows
v_returns	117 rows
v_tickets	112 rows
v_promotions	1 row

JDBC

Agents query it

Retriever (SQL templates)

Evidence Evaluator (re-queries)

Traceability Check (re-runs cited SQL)

Live, 26 Oct 2026
3 demo scenarios over Zetaris JDBC
+ Nightshift: 72 / 72 receipts OK

Public Olist dataset
federated in container olist (CC BY-NC-SA 4.0)
27 / 27 claims green; 1-row mutation → red

SAMPLE_MODE fallback
same view names, same SQL on a DuckDB snapshot; memo says “SAMPLE MODE: no Zetaris”

Green = runs in the product today · Amber = benchmark harness only

Meterless runs every model call, with a receipt

Nightshift 1.2.7 is the primary model provider; H-MEM and Markovian run through a Node bridge

Nightshift 1.2.7
primary model provider; pinned S1 / S2 pin check at start, receipt per call

H-MEM
hypotheses, retrieval calls, evidence, support / refute edges + trust ledger

Markovian
one chunk per Evaluator → Planner round (max 3), then [TASK_COMPLETE]

agent	tier	routed_model	latency_s	ok	pin_ok	client_request_id
planner	s2	nvidia/nemotron-3-ultra-550b-a55b	3.29 s	true	true	9106722d…
retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.45 s	true	true	075c0f95…
s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.37 s	true	true	5aa93a3c…
evidence_evaluator	s1	nvidia/nemotron-3.5-lightning-30b-a3b	0.74 s	true	true	04b0fd7e…
cause_analyst	s2	nvidia/nemotron-3-ultra-550b-a55b	2.86 s	true	true	9542bd34…
writer	s2	nvidia/nemotron-3-ultra-550b-a55b	2.80 s	true	true	f1d75927…

Real rows (first call per agent) from eval/nightshift_live/v2_agent_ids/zetaris-receipts-example-002.jsonl: live Zetaris + Nightshift, 26 Oct 2026, planted-trap run. 25 / 25 receipts OK, fallback never triggered. Ids truncated.

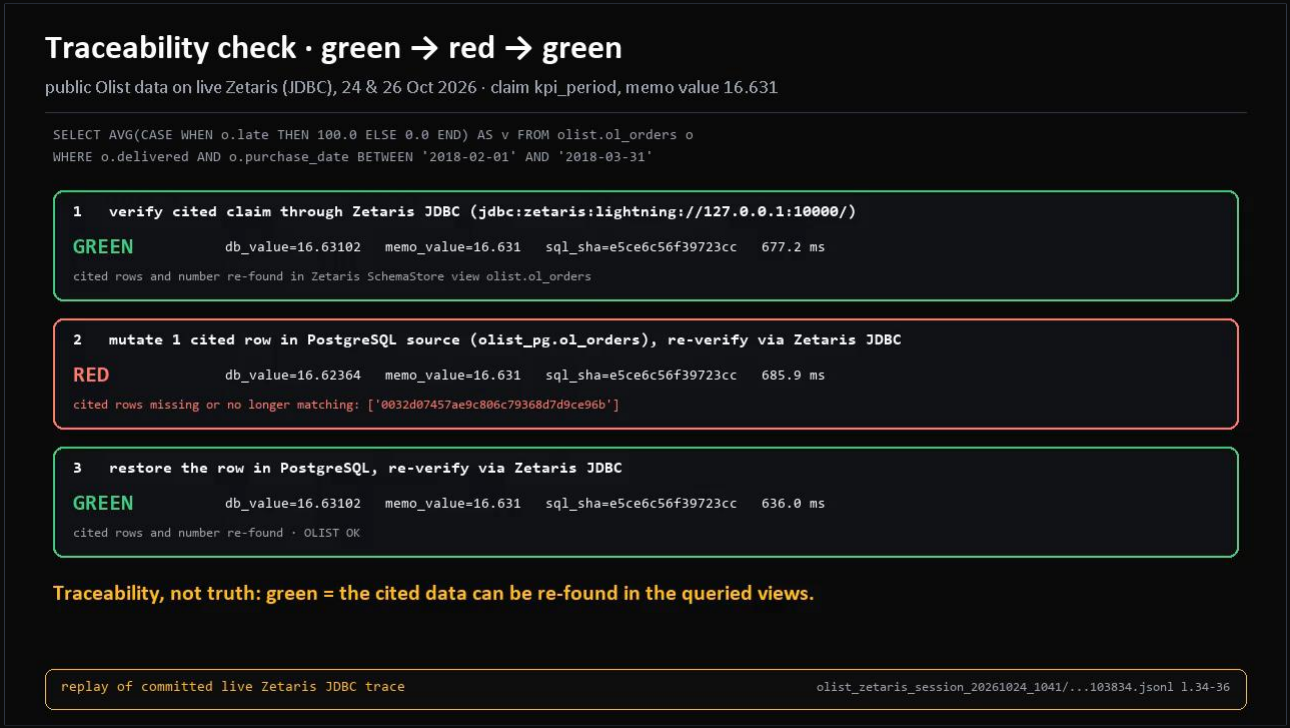
calls in this run: planner 1 · retriever 9 · s1_filter 6 · evidence_evaluator 5 · cause_analyst 1 · writer 1 · pin checks 2

All 3 live scenarios:
72 / 72 receipts OK, 0 failed

Green = runs in the product today · Amber = benchmark harness only

Change one cited row and the claim turns red

The Traceability Check re-runs the cited query through Zetaris and re-finds the cited rows and numbers



GREEN
claim cites rows; re-run via Zetaris; rows + number re-found

↓

RED
1 cited row mutated in PostgreSQL: “cited rows missing or no longer matching”

↓

GREEN
row restored; re-run via Zetaris; claim passes again

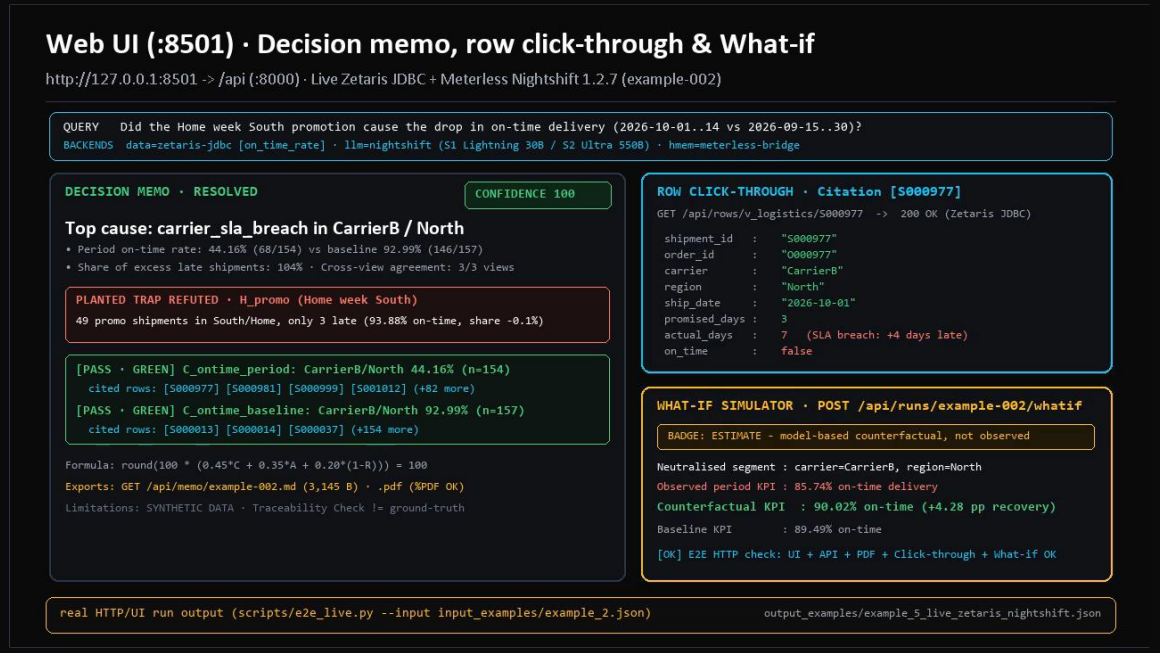
`Traceability, not truth.`

Green means the cited data can be re-found in the queried views, not that the claim is true.

Clip: replay of live Zetaris JDBC trace on public Olist data (24 Oct). Also verified: verify_live_demo.py --backend zetaris (DEMO OK), olist_demo.py --mutate (OLIST OK).

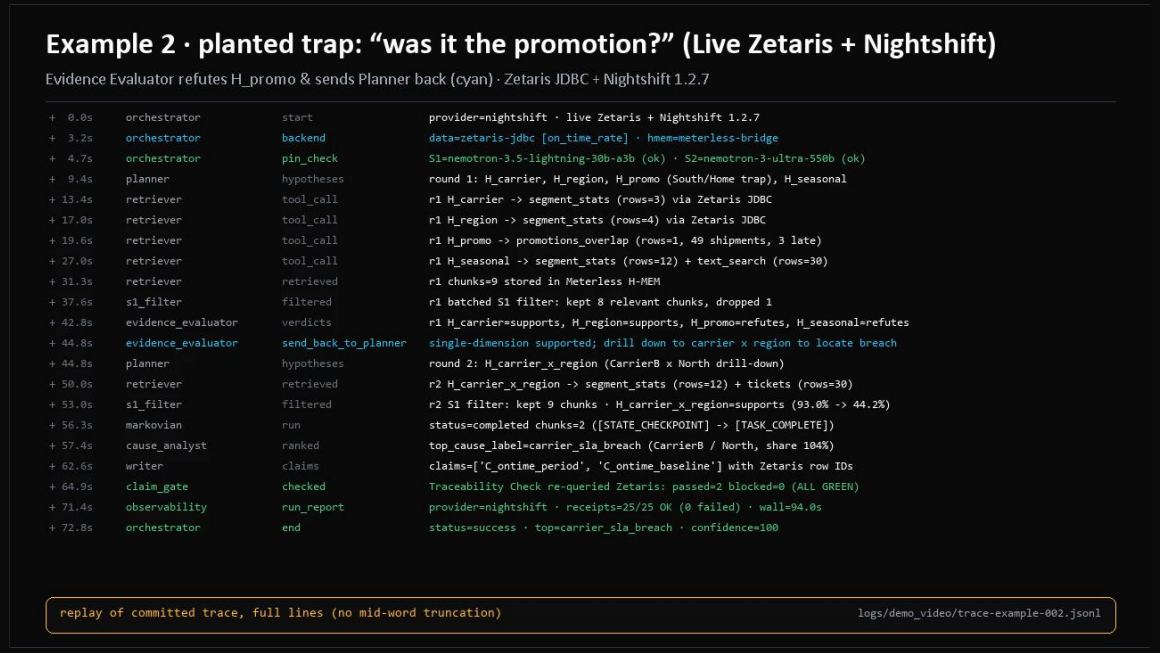
What the analyst sees: a memo, and the trace behind it

Web UI (:8501) over the FastAPI service (:8000); every run writes a JSONL trace



Web UI (:8501) over FastAPI (:8000): decision memo with [PASS · GREEN] claims, row-level click-through to Zetaris JDBC, and What-if counterfactual simulator.

`output_examples/example_5_live_zetaris_nightshift.json · Live Zetaris + Nightshift`



Replay of the committed trace for the planted-trap example: hand-offs, the Evaluator sending the Planner back (cyan), claims passing the check (green).

`logs/demo_video/trace-example-002.jsonl · Live Zetaris JDBC + Nightshift 1.2.7`

Technical stack: plain Python, no agent framework

Own orchestrator (app/pipeline.py) and own agentic retriever; numbers always come from SQL

interface	FastAPI API :8000 · web UI :8501 · CLI run.py · memo PDF (fpdf2)
orchestration	own Python orchestrator · own agentic retriever (SQL templates + BM25 / dense)
models	NVIDIA Nemotron 3.5 Lightning 30B (S1) + Nemotron 3 Ultra (S2) via Meterless Nightshift 1.2.7
memory & rounds	Meterless H-MEM + Markovian (vendored, Apache-2.0) via Node bridge
data	Zetaris Lightning over JDBC (JayDeBeApi) · DuckDB snapshot for SAMPLE_MODE
ops & evidence	Docker compose · pytest + GitHub CI · JSONL traces, receipts, H-MEM ledgers

MIT licence · secrets only in a local .env (gitignored) · secret scan clean

Evidence it works: a pre-registered benchmark run

TableBench, n = 361, single run, no tuning (docs/V11_RESULTS.md) · benchmark lane, not the live KPI pipeline

B · BENCHMARK-ONLY: bench/run.py · V7 answers + V9_full confidence



+15.5 pp (95% CI +11.1 to +20.2)

semantic accuracy, paired bootstrap

0.834
V9_full confidence AUROC

36.6% → 77.3%
answers 36.6% of questions, 77.3% accurate when it answers

8.3%
wrong answers given, of all questions (target ≤ 10%)

+0.199 vs −0.346
net score (+1 right, −1 wrong, 0 abstain): V7+V9 vs RAG

The live KPI memo does not use V7 / V9: its confidence is the fixed formula in app/confidence.py.

Honest limits, and what comes next

All numbers from the README Status section and the linked reports

limits

traceability ≠ correctness
140 of 171 wrong test500 answers passed the check

dev 21/22 vs plain RAG 12/22 is in-sample
deterministic rules alone also score 21/22; plain RAG is below the trivial rule (13/22)

V9 confidence on hard sets
not better than cheap signals; conformal guarantee fails on DocFinQA

narrative check over-flags
173 of 400 correct answers flagged (43%)

abstains often
63% of TableBench questions get no answer

next steps

- sign / scale / range checks on numbers
- Markovian control in the app (today: benchmark only)
- a non-author evaluation set
- more KPI families

Green = runs in the product today · Amber = benchmark harness only

AI usage, pre-existing work, and how to run it

Declared in README “AI and Model Usage” and docs/PRE_EXISTING_WORK.md

AI and Model Usage

Runtime: Nemotron 3.5 Lightning 30B (S1) and Nemotron 3 Ultra (S2) through Meterless Nightshift, pinned and receipted per call.

Code: written during the build window by AI coding agents (Grok Bot) under the author’s direction; reviewed, tested and committed by the author.

Cursor was not used to write this code; the organisers’ announcement of 22 Oct 2026 made it optional.

Pre-existing work

Environment setup (Zetaris platform, API keys), a throwaway connectivity scaffold not in this repo, design material (diagrams v2–v6.2, architecture documents) and the empty repository skeleton.

Open-source Meterless engines (Apache-2.0) and the public Olist dataset (CC BY-NC-SA 4.0) used under their licences.

Run it (no credentials)

```
git clone https://github.com/
AlonsoAlviraa/veriagent-why
cd veriagent-why
pip install -r requirements.txt
python run.py --input \
  input_examples/example_1.json



SAMPLE_MODE fallback: DuckDB, no model
```

Thanks

Zetaris · Meterless · NVIDIA
Olist data (CC BY-NC-SA 4.0) · MIT licence

github.com/AlonsoAlviraa/veriagent-why

appendix

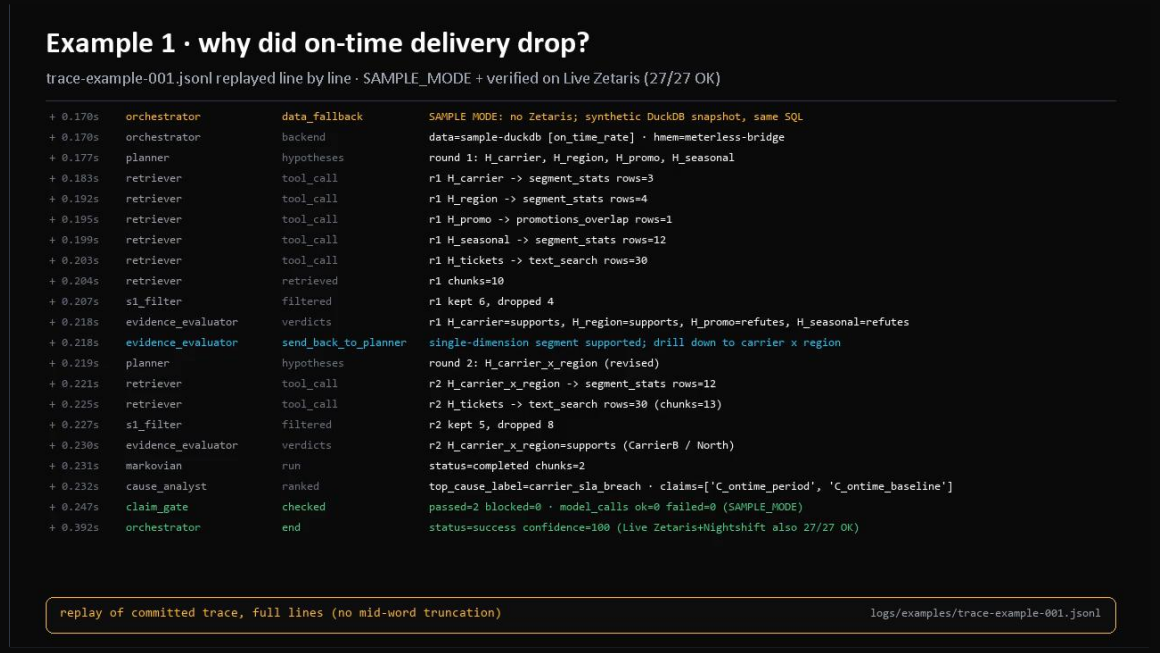
Everything generated

replays of committed traces, receipts and results · recordings to add · file list

Every clip is a replay of a committed file, not a live screen recording.

Replays: example 1 and the honest “unresolved” case

Committed traces in logs/examples/ (SAMPLE_MODE, synthetic data)

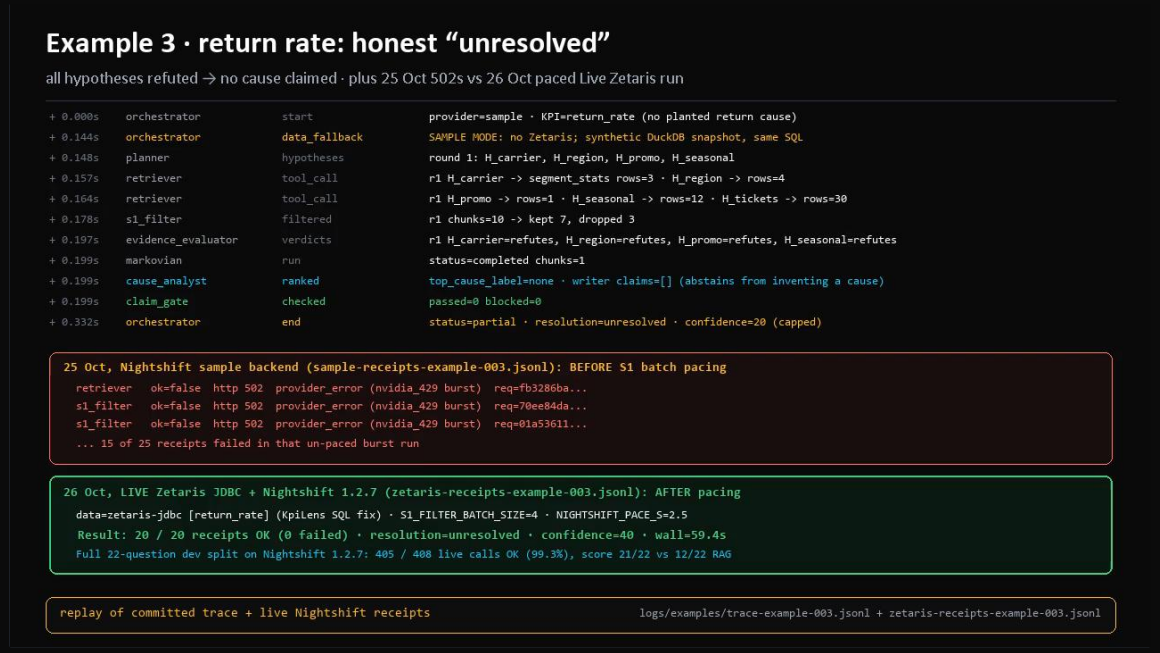


Example 1 · why did on-time delivery drop?

trace-example-001.jsonl replayed line by line · SAMPLE_MODE + verified on Live Zetaris (27/27 OK)

+ 0.170s	orchestrator	data_fallback	SAMPLE MODE: no Zetaris; synthetic DuckDB snapshot, same SQL
+ 0.170s	orchestrator	backend	data=sample-duckdb [on_time_rate] · hmem=meterless-bridge
+ 0.177s	planner	hypotheses	round 1: H_carrier, H_region, H_promo, H_seasonal
+ 0.183s	retriever	tool_call	r1 H_carrier -> segment_stats rows=3
+ 0.192s	retriever	tool_call	r1 H_region -> segment_stats rows=4
+ 0.195s	retriever	tool_call	r1 H_promo -> promotions_overlap rows=1
+ 0.199s	retriever	tool_call	r1 H_seasonal -> segment_stats rows=12
+ 0.203s	retriever	tool_call	r1 H_tickets -> text_search rows=30
+ 0.204s	retriever	retrieved	r1 chunks=10
+ 0.207s	s1_filter	filtered	r1 kept 6, dropped 4
+ 0.218s	evidence_evaluator	verdicts	r1 H_carrier=supports, H_region=supports, H_promo=refutes, H_seasonal=refutes
+ 0.218s	evidence_evaluator	send_back_to_planner	single-dimension segment supported; drill down to carrier x region
+ 0.219s	planner	hypotheses	round 2: H_carrier_x_region (revised)
+ 0.221s	retriever	tool_call	r2 H_carrier_x_region -> segment_stats rows=12
+ 0.225s	retriever	tool_call	r2 H_tickets -> text_search rows=30 (chunks=13)
+ 0.227s	s1_filter	filtered	r2 kept 5, dropped 8
+ 0.230s	evidence_evaluator	verdicts	r2 H_carrier_x_region=supports (CarrierB / North)
+ 0.231s	markovian	run	status=completed chunks=2
+ 0.232s	cause_analyst	ranked	top_cause_label=carrier_sla_breach · claims=['C_ontime_period', 'C_ontime_baseline']
+ 0.247s	claim_gate	checked	passed=2 blocked=0 · model_calls ok=0 failed=0 (SAMPLE_MODE)
+ 0.392s	orchestrator	end	status=success confidence=100 (Live Zetaris+Nightshift also 27/27 OK)

replay of committed trace, full lines (no mid-word truncation) — logs/examples/trace-example-001.jsonl



Example 3 · return rate: honest “unresolved”

all hypotheses refuted → no cause claimed · plus 25 Oct 502s vs 26 Oct paced Live Zetaris run

+ 0.000s	orchestrator	start	provider=sample · KPI=return_rate (no planted return cause)
+ 0.144s	orchestrator	data_fallback	SAMPLE MODE: no Zetaris; synthetic DuckDB snapshot, same SQL
+ 0.148s	planner	hypotheses	round 1: H_carrier, H_region, H_promo, H_seasonal
+ 0.157s	retriever	tool_call	r1 H_carrier -> segment_stats rows=3 · H_region -> rows=4
+ 0.164s	retriever	tool_call	r1 H_promo -> rows=1 · H_seasonal -> rows=12 · H_tickets -> rows=30
+ 0.178s	s1_filter	filtered	r1 chunks=10 -> kept 7, dropped 3
+ 0.197s	evidence_evaluator	verdicts	r1 H_carrier=refutes, H_region=refutes, H_promo=refutes, H_seasonal=refutes
+ 0.199s	markovian	run	status=completed chunks=1
+ 0.199s	cause_analyst	ranked	top_cause_label=none · writer claims=[] (abstains from inventing a cause)
+ 0.199s	claim_gate	checked	passed=0 blocked=0
+ 0.332s	orchestrator	end	status=partial · resolution=unresolved · confidence=20 (capped)

25 Oct, Nightshift sample backend (sample-receipts-example-003.jsonl): BEFORE S1 batch pacing

retriever ok=false http 502 provider_error (nvidia_429 burst) req=fb3286ba...
s1_filter ok=false http 502 provider_error (nvidia_429 burst) req=70ee84da...
s1_filter ok=false http 502 provider_error (nvidia_429 burst) req=01a53611...
... 15 of 25 receipts failed in that un-paced burst run

26 Oct, LIVE Zetaris JDBC + Nightshift 1.2.7 (zetaris-receipts-example-003.jsonl): AFTER pacing

data=zetaris-jdbc [return_rate] (KpiLens SQL fix) · S1_FILTER_BATCH_SIZE=4 · NIGHTSHIFT_PACE_S=2.5
Result: 20 / 20 receipts OK (0 failed) · resolution=unresolved · confidence=40 · wall=59.4s
Full 22-question dev split on Nightshift 1.2.7: 405 / 408 live calls OK (99.3%), score 21/22 vs 12/22 RAG

replay of committed trace + live Nightshift receipts — logs/examples/trace-example-003.jsonl + zetaris-receipts-example-003.jsonl

Example 1, on-time drop: two rounds, send-back to the Planner, carrier_sla_breach, confidence 100. Source: logs/examples/trace-example-001.jsonl (16 s).

Example 3, return rate: every hypothesis refuted, no cause claimed (confidence 20 sample / 40 live). Also shows the 15 of 25 failed calls of the 25 Oct run, fixed by pacing: 20/20 OK live on 26 Oct (16.5 s).

Replays: per-call receipts and the benchmark result

Live receipts from 26 Oct 2026 · V11 numbers from docs/V11_RESULTS.md

Nightshift receipts · live Zetaris + Nightshift 1.2.7, 26 Oct 2026

planted-trap run (example 2): one receipt per model call, agent → pinned Nemotron model

time (CET)	agent	tier	routed_model	lat_s	ok	pin	req_id
18:16:38	(pin_check)	s1	nvidia/nemotron-3.5-lightning-30b-a3b	0.54s	true	true	c1bea8fb
18:16:40	(pin_check)	s2	nvidia/nemotron-3-ultra-550b-a55b	2.46s	true	true	ac4bbcd4
18:16:48	planner	s2	nvidia/nemotron-3-ultra-550b-a55b	3.29s	true	true	9106722d
18:16:51	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.45s	true	true	075c0f95
18:16:54	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.90s	true	true	ac3b0d05
18:16:57	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.40s	true	true	bc2c4844
18:17:00	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.90s	true	true	d08febd9
18:17:03	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.45s	true	true	1312c087
18:17:06	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.73s	true	true	30cc7912
18:17:09	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.74s	true	true	3979313d
18:17:14	s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.37s	true	true	5aa93a3c
18:17:16	s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.47s	true	true	cb647690
18:17:18	s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.44s	true	true	8798b45e
18:17:21	evidence_evaluator	s1	nvidia/nemotron-3.5-lightning-30b-a3b	0.74s	true	true	04b0fd7e
18:17:23	evidence_evaluator	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.28s	true	true	0f5ff539
18:17:25	evidence_evaluator	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.29s	true	true	7df73c9a
18:17:28	evidence_evaluator	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.25s	true	true	9de2d2b4
18:17:33	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.81s	true	true	a98f6865
18:17:41	retriever	s1	nvidia/nemotron-3.5-lightning-30b-a3b	7.61s	true	true	6be26639
18:17:45	s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.48s	true	true	2aa1b1ef
18:17:48	s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.40s	true	true	bb841cff
18:17:50	s1_filter	s1	nvidia/nemotron-3.5-lightning-30b-a3b	2.49s	true	true	c0c29da3
18:17:53	evidence_evaluator	s1	nvidia/nemotron-3.5-lightning-30b-a3b	1.79s	true	true	c784c3dc
18:17:58	cause_analyst	s2	nvidia/nemotron-3-ultra-550b-a55b	2.86s	true	true	9542bd34
18:18:02	writer	s2	nvidia/nemotron-3-ultra-550b-a55b	2.80s	true	true	f1d75927

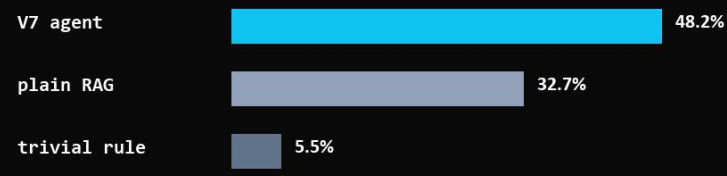
25/25 receipts OK · All 3 live Zetaris scenarios: 72/72 receipts OK (0 failed) · no secrets stored

replay of committed live receipts — eval/nightshift_live/v2_agent_ids/zetaris-receipts-example-002.jsonl

All 25 receipts of the live planted-trap run: agent, tier, pinned Nemotron model, latency, ok, pin_ok, request id (truncated). Source: eval/nightshift_live/v2_agent_ids/zetaris-receipts-example-002.jsonl (15.6 s).

V11 · TableBench, n = 361, pre-registered, single run

benchmark lane only (bench/run.py) · not the live KPI pipeline · verdict: WORKS (all A1–A4 pass)



+15.5 pp vs plain RAG (95% CI +11.1 to +20.2) · paired bootstrap

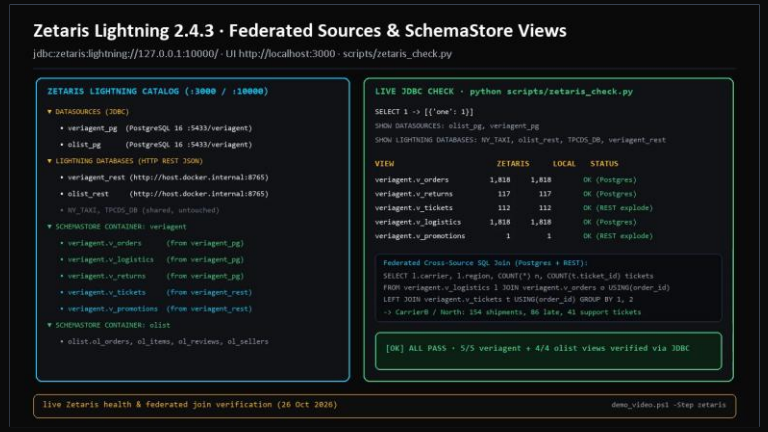
0.834	36.6%	77.3%
V9_full confidence AUROC [0.790, 0.874]	of questions answered	accuracy when it answers
8.3% ≤ 10%	+0.199 vs −0.346	A1–A4 PRE-REGISTERED VERDICT: WORKS (4/4 PASS)
realized risk (conformal α = 0.10)	net score V7+V9 vs plain RAG	

replay of committed results, pre-registered in docs/V11_PREREG.md — docs/V11_RESULTS.md

V11 TableBench (n = 361, pre-registered, single run): V7 48.2% vs plain RAG 32.7%, AUROC 0.834, 77.3% accurate at 36.6% coverage. Benchmark lane only (16 s).

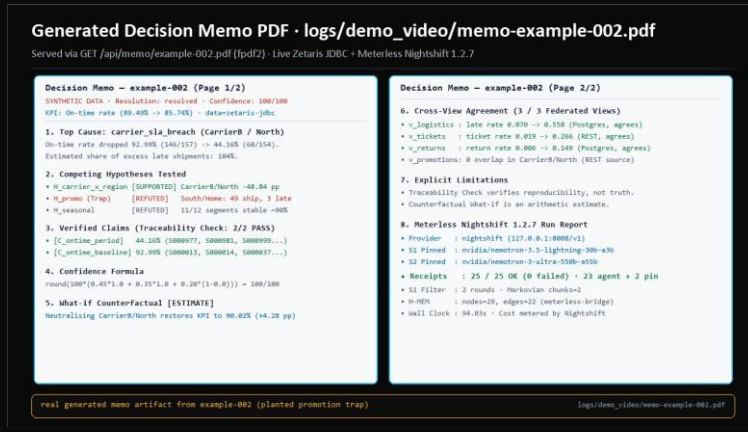
Live machine clips: Zetaris, memo PDF, and mutation

Captured on Alonso's PC (UI :8501 · API :8000 · Zetaris :3000/:10000 · PostgreSQL :5433 · REST :8765)



Zetaris Lightning 2.4.3 catalog & live JDBC check: veriagent_pg (PostgreSQL 16) + veriagent_rest (HTTP JSON) federated into 5 SchemaStore views and verified via cross-source SQL join.

`scripts/zetaris_check.py · 5/5 veriagent + 4/4 olist views OK`



Generated 2-page Decision Memo PDF for example-002: refuted promotion trap (93.88% on-time), 3/3 cross-view agreement, and 25/25 Meterless Nightshift receipts.

`GET /api/memo/example-002.pdf · logs/demo_video/memo-example-002.pdf`



Live Zetaris JDBC mutation check: mutating 5 cited rows in PostgreSQL flips claim C_ontime_period GREEN (44.16%) → RED (50.65% drift) → GREEN (44.16% restored).

`demo_video.ps1 -Step mutate · logs/trace-verify-demo-9eb5df.jsonl`

All 3 clips above are embedded from live executions on this machine (Zetaris JDBC :10000, FastAPI :8000, Web UI :8501, Meterless Nightshift :8088).

Everything generated: files in the repository

github.com/AlonsoAlviraa/veriagent-why (main @ ee3db6a)

outputs & runs

output_examples/example_1..3.json
  SAMPLE_MODE outputs
output_examples/example_4_live_nightshift.json
output_examples/example_5_live_zetaris_nightshift.json
logs/examples/trace-example-00{1,2,3}.jsonl
logs/examples/memo-example-00{1,2,3}.md
logs/examples/hmem-ledger-example-00{1,2,3}.jsonl

live sessions & receipts

eval/nightshift_live/v2_agent_ids/
  zetaris-receipts-example-00{1,2,3}.jsonl
  sample-receipts-example-00{1,2,3}.jsonl
logs/olist_zetaris_session_20261024_1041/
  traces, results, memos olist_q1..q3
logs/zetaris_session_20261024_1015/
docs/OLIST_ZETARIS_SESSION_20261024.md

evaluation & benchmarks

eval/RESULTS_dev_nightshift.md
eval/RESULTS_dev_nvidia.md, RESULTS_dev_sample.md
docs/V11_RESULTS.md + docs/v11/MANIFEST.sha256
docs/V10_RESULTS.md, TEST500_RESULTS.md
docs/V9_GENERALIZATION_RESULTS.md
docs/TEST_BATTERY_REPORT.md
docs/trace_audit/TEST500_ERRORS.md

Also: docs/diagrams/veriagent_why_arch_FINAL.png · docs/PRE_EXISTING_WORK.md · README (setup, AI and Model Usage, limits) · clips in this deck replay these files.

Green = runs in the product today · Amber = benchmark harness only